

这个项目最值得学习的不是界面效果，而是它已经形成了一条完整的“剧情脚本编译与执行流水线”：

```
剧情文本
→ Lexer 词法分析
→ Token 列表
→ Parser 语法分析
→ SyntaxTree
→ Transpiler 转换
→ DialogueTree
→ ScenePlayer 执行
→ TextBox / CharacterDisplayer 表现
```

这条流水线与你计划中的 DrameX 高度接近。因此建议不要从第一行开始顺序读代码，而应沿着“一句对白如何最终显示出来”这条调用链学习。

项目实际入口位于：

[godot/project.godot (</D:/projects/godot-projects/Resources/Code/godot-4-2d-visual-novel/godot/project.godot>)

需要打开的是 `godot` 目录里的项目，而不是仓库根目录或 `start-project`。本地版本配置为 Godot 4.6。

一、先建立项目的整体地图

主要模块如下：

```
Main
├─ 加载所有剧情文本
├─ 调用编译流水线
└─ 创建 ScenePlayer

Parser/
├─ SceneLexer      文本 → Token
├─ SceneParser     Token → SyntaxTree
└─ SceneTranspiler SyntaxTree → DialogueTree

ScenePlayer
├─ 维护当前剧情节点
├─ 执行命令
├─ 等待玩家操作或动画完成
└─ 将表现工作委托给子节点

ScenePlayer.tscn
├─ Background
├─ CharacterDisplayer
├─ TextBox
├─ FadeAnimationPlayer
└─ BGM

Autoloads/
├─ ResourceDB      根据 ID 查找角色、背景、音乐
└─ Variables       保存剧情变量
```

其中最核心的分界是：

- Parser 系列处理“语言”
- `ScenePlayer` 处理“运行”
- `TextBox` 等节点处理“显示”
- Resource 类处理“数据”

二、第一步：先运行和修改剧情，不读解析器

先打开：

[Main.tscn (</D:/projects/godot-projects/Resources/Code/godot-4-2d-visual-novel/godot/Main.tscn>)]

该场景导出了一个剧情文件数组：

```
scripts = [  
    "res://Parser/test-scene.txt",  
    "res://Parser/test-scene2.txt"  
]
```

然后阅读：

[test-scene.txt (</D:/projects/godot-projects/Resources/Code/godot-4-2d-visual-novel/godot/Parser/test-scene.txt>)]

里面涵盖了项目的大部分语法：

```
set started_over false  
mark jump_location  
  
background dani_bedroom  
  
sophia happy enter left "Hi there!"  
  
choice:  
    "Start over":  
        ...
```

先做这些小实验：

1. 修改一行对白。
2. 改变角色表情。
3. 将角色从 `left` 改到 `right`。
4. 添加一个新选项。
5. 添加一个 `mark` 和 `jump`。
6. 添加一个布尔变量和条件分支。
7. 添加一个背景切换。

每次只修改一处并运行。这个阶段的目标不是研究编译器，而是建立：

剧情写法 → 屏幕效果

之间的直觉。

建议记录一张语法表

脚本写法	产生的行为
"文本"	旁白
sophia "文本"	角色对白
sophia happy enter left "文本"	角色、表情、动画、位置、对白
background id	切换背景
song id	播放音乐
transition fade_out	播放转场
choice:	创建选择分支
mark name	创建跳转点
jump name	跳转
set name value	设置持久变量

这一步预计一到两个学习时段。

三、第二步：跟踪一句对白的完整调用链

以这句为例：

```
sophia happy enter left "Hi there!"
```

1. Main 加载并编译文件

阅读：

[Main.gd:17 (</D:/projects/godot-projects/Resources/Code/godot-4-2d-visual-novel/godot/Main.gd:17>)]

核心流程是：

```
var text := lexer.read_file_content(script)
var tokens := lexer.tokenize(text)
var tree := parser.parse(tokens)
var dialogue := transpiler.transpile(tree, 0)
```

可以将其理解为：

源代码

- 单词和符号
- 有层级的语法结构
- 可以执行的节点图

Main 把编译后的所有剧情存入 SCENES，然后调用 _play_scene(0)。

2. Main 创建播放器

阅读：

[Main.gd:37 (</D:/projects/godot-projects/Resources/Code/godot-4-2d-visual-novel/godot/Main.gd:37>)]

它会：

1. 删除旧的 `ScenePlayer`。
2. 实例化新的 `ScenePlayer.tscn`。
3. 注入编译后的 `DialogueTree`。
4. 连接结束和重新开始信号。
5. 调用 `run_scene()`。

这里体现了一个不错的边界：

Main 管理多段剧情的生命周期
ScenePlayer 播放单段剧情

3. ScenePlayer 执行对白节点

阅读：

[ScenePlayer.gd:27 (</D:/projects/godot-projects/Resources/Code/godot-4-2d-visual-novel/godot/ScenePlayer.gd:27>)]

核心是：

```
var key = 0

while key != KEY_END_OF_SCENE:
    var node = _scene_data[key]
    ...
    key = node.next
```

它不是简单的数组播放，而是在运行一个节点图：

节点 0 → 节点 1 → 节点 2
 ↘ 选择分支 A
 ↘ 选择分支 B

普通对白节点包含：

```
line
character
expression
animation
side
next
```

`ScenePlayer` 根据角色 ID 从 `ResourceDB` 得到角色，再调用：

```
_character_displayer.display(...)
_text_box.display(...)
await _text_box.next_requested
```

这里的 `await` 非常值得学习。播放器暂停在这一行，直到玩家完成阅读并请求下一句，然后才更新 `key`。

四、第三步：学习场景组合和信号

打开：

[ScenePlayer.tscn (</D:/projects/godot-projects/Resources/Code/godot-4-2d-visual-novel/godot/ScenePlayer.tscn>)]

重点观察：

```
ScenePlayer
├─ Background
├─ CharacterDisplayer
├─ TextBox
├─ ColorRect
├─ FadeAnimationPlayer
└─ BGM
```

这个场景是一个“视觉小说舞台”：

- 背景只负责背景
- 角色显示器只负责角色
- 文本框只负责对白和选择
- AnimationPlayer 只负责转场
- AudioStreamPlayer 只负责音乐
- `ScenePlayer.gd` 负责协调

这是你自己的项目应该学习的主要部分。

重点跟踪三个信号

`TextBox.next_requested`

[TextBox.gd:46 (</D:/projects/godot-projects/Resources/Code/godot-4-2d-visual-novel/godot/TextBox/TextBox.gd:46>)]

玩家按键时：

- 如果文字尚未显示完，就完成文字动画。
- 如果文字已经显示完，就发出 `next_requested`。

`ScenePlayer` 使用：

```
await _text_box.next_requested
```

等待该信号。

`ChoiceSelector.choice_made`

[ChoiceSelector.gd (</D:/projects/godot-projects/Resources/Code/godot-4-2d-visual-novel/godot/TextBox/ChoiceSelector.gd>)]

选择器并不知道整个剧情，只返回被选择分支的目标节点：

```
choice_made.emit(target_id)
```

信号经过 `TextBox` 转发给 `ScenePlayer`。

`CharacterDisplayer.display_finished`

[CharacterDisplayer.gd:35 (</D:/projects/godot-projects/Resources/Code/godot-4-2d-visual-novel/godot/Characters/CharacterDisplayer.gd:35>)]

角色进入或离开动画完成后发出信号。播放器可以：

```
await _character_displayer.display_finished
```

这体现了很实用的模式：

上层发命令，下层做动画；下层用信号报告完成，上层继续流程。

推荐练习

暂时不要修改解析器。给 `CharacterDisplayer` 增加一个概念性动画，例如：

```
shake  
flash  
move_center
```

完整思考以下接口：

```
display(character, side, expression, animation)
```

尝试回答：

- 新动画属于角色显示器还是剧情播放器？
- 动画完成由谁通知？
- `ScenePlayer` 是否需要知道 Tween 的细节？

正确方向是：Tween 留在显示器内，播放器只知道动画名称和完成信号。

五、第四步：学习数据与表现的分离

阅读：

[Character.gd (</D:/projects/godot-projects/Resources/Code/godot-4-2d-visual-novel/godot/Characters/Character.gd>)]

它继承 `Resource`，只保存角色数据：

```
@export var id  
@export var display_name  
@export var bio  
@export var default_image  
@export var images
```

实际角色实例位于：

[sophia.tres (</D:/projects/godot-projects/Resources/Code/godot-4-2d-visual-novel/godot/Characters/sophia.tres>)]

大致对应：

```
id = sophia
display_name = Sophia
images = {
    neutral: ...
    happy: ...
    angry: ...
}
```

然后：

[CharacterDisplayer.gd (</D:/projects/godot-projects/Resources/Code/godot-4-2d-visual-novel/godot/Characters/CharacterDisplayer.gd>)]

负责把 `Character` 数据显示出来。

这个分离非常适合你的项目：

```
Character Resource
    角色是谁、拥有哪些资源

CharacterDisplayer
    如何显示角色

StoryRunner
    何时要求显示角色
```

推荐练习：添加一个新角色

完整走一遍：

1. 新建角色贴图。
2. 新建 `.tres`。
3. 设置唯一 `id`。
4. 填写默认表情和其他表情。
5. 在剧情文件中用 ID 调用角色。
6. 不修改 `Character.gd` 和 `ResourceDB.gd`。

如果能够只通过资源和剧情文件添加角色，就说明你理解了数据驱动设计。

这正是它比你当前项目中 `Character.CHARACTER_DETAILS` 更值得学习的地方。

六、第五步：学习 ResourceDB

阅读：

[ResourceDB.gd (</D:/projects/godot-projects/Resources/Code/godot-4-2d-visual-novel/godot/Autoloads/ResourceDB.gd>)]

它在启动时扫描：

```
Characters/
Backgrounds/
Sound/BGM/
```

把找到的 `.tres` 按资源 ID 放入字典：

```
characters["sophia"] → sophia.tres
backgrounds["dani_bedroom"] → dani_bedroom.tres
```

播放器只使用逻辑 ID：

```
ResourceDB.get_character("sophia")
ResourceDB.get_background("dani_bedroom")
```

而不用硬编码：

```
load("res://Characters/Sophia/sophia_happy.png")
```

这与你计划中的 DrameX 自动资源发现很接近。

应该学习的部分

- 用 Resource 表达领域数据。
- 用稳定 ID 隔离剧情和文件路径。
- 在一个地方建立资源索引。
- 对外提供查询方法，不直接暴露字典。

不要照搬的部分

它只扫描固定目录下第一层 `.tres`，不支持复杂的嵌套目录。

另外 `_load_resources()` 有一个控制流隐患：

```
if not call(check_type_function, resource):
    continue
```

`continue` 发生前没有执行 `directory.get_next()`。如果目录里出现类型不匹配的 `.tres`，循环可能一直停在同一个文件。

因此你应学习其“资源注册表”思想，但不要直接复制实现。

七、第六步：最后再学习编译器

Parser 是项目最复杂的部分，应当最后阅读。

1. Lexer：文本变成 Token

阅读：

[SceneLexer.gd:137 (</D:/projects/godot-projects/Resources/Code/godot-4-2d-visual-novel/godot/Parser/SceneLexer.gd:137>)]

Lexer 不理解完整剧情含义，只识别：

- 命令
- 符号
- 字符串
- 换行

- 缩进开始和结束
- `choice`
- `if/elif/else`
- 注释

例如：

```
sophia happy enter left "Hi there!"
```

概念上会变成：

```
SYMBOL(sophia)
SYMBOL(happy)
SYMBOL(enter)
SYMBOL(left)
STRING_LITERAL(Hi there!)
NEWLINE
```

重点阅读：

- `DialogueScript` 如何维护当前位置。
- `tokenize()` 如何逐字符分派。
- `_tokenize_symbol()` 如何读取普通标识符。
- `_tokenize_string_literal()` 如何读取引号字符串。
- 缩进如何形成 `BEGIN_BLOCK` 和 `END_BLOCK`。

Lexer 练习

为每种输入手工写出你预期的 Token：

```
background room
"旁白"
choice:
    "Yes":
        "Continue"
```

然后在 `ParserTest` 中打印实际 Token，对比预期。

2. Parser: Token 变成 SyntaxTree

阅读：

[SceneParser.gd:348 (</D:/projects/godot-projects/Resources/Code/godot-4-2d-visual-novel/godot/Parser/SceneParser.gd:348>)]

Parser 把线性的 Token 恢复成结构：

```
ChoiceExpression
├─ ChoiceBlock "Yes"
│   └─ DialogueExpression
└─ ChoiceBlock "No"
    └─ JumpExpression
```

重点理解这些类：

- SyntaxTree
- BaseExpression
- FunctionExpression
- ChoiceBlockExpression
- ConditionalExpression
- ConditionalTreeExpression

Lexer 只回答“这是什么词”，Parser 回答“这些词构成什么语句”。

Parser 练习

拿一个两选项的最小脚本：

```
choice:
  "A":
    "Selected A"
  "B":
    "Selected B"
```

在纸上画出它的语法树。不要急着研究全部条件逻辑。

3. Transpiler: 语法树变成可执行图

阅读：

[SceneTranspiler.gd:201 (</D:/projects/godot-projects/Resources/Code/godot-4-2d-visual-novel/godot/Parser/SceneTranspiler.gd:201>)]

它定义了运行时节点类型：

- DialogueNode
- BackgroundCommandNode
- SongCommandNode
- TransitionCommandNode
- ChoiceTreeNode
- ConditionalTreeNode
- SetCommandNode
- JumpCommandNode
- MarkCommandNode
- PassCommandNode

例如语法树里的对白会通过：

[SceneTranspiler.gd:404 (</D:/projects/godot-projects/Resources/Code/godot-4-2d-visual-novel/godot/Parser/SceneTranspiler.gd:404>)]

转换成：

```
DialogueNode
├─ line
├─ character
├─ expression
└─ animation
```

```
└─ side
└─ next
```

这里实际就是你未来 DrameX 所需的“中间表示”。

为什么需要 Transpiler

Parser 得到的是“作者写了什么结构”：

```
choice
└─ A 的缩进块
└─ B 的缩进块
```

播放器需要的是“下一步跳到哪里”：

```
节点 5: 显示选择
└─ A → 节点 1000000006
└─ B → 节点 1000000010
```

Transpiler 负责把嵌套结构转换成带跳转地址的执行图。

八、第七步：学习异步剧情执行

`ScenePlayer.run_scene()` 是整个项目运行时最值得研究的函数。

可以把它改写为伪代码：

```
key = 0

while key != END:
    node = nodes[key]

    如果是背景命令：
        更换背景

    如果带角色：
        显示角色

    如果是对白：
        显示文字
        等待玩家
        key = node.next

    否则如果是选择：
        显示选择
        等待选择结果
        key = 选择目标

    否则如果是条件：
        计算条件
        key = 对应分支

    否则：
        key = node.next
```

你需要重点理解：

`await` 是流程控制的一部分

```
await _text_box.next_requested
```

表示剧情解释器暂停，不是整个游戏卡住。

```
key = await _text_box.choice_made
```

表示选择结果直接决定下一执行节点。

这种写法非常适合视觉小说，因为剧本天然就是：

```
执行动作
→ 等待玩家或动画
→ 继续执行
```

`key` 是程序计数器

它相当于 CPU 的 instruction pointer：

```
当前执行到哪条剧情指令
```

理解这一点后，你会更容易设计：

- 存档和读档
- `goto`
- `call/return`
- 证言浏览
- 调查模式暂停与恢复

九、这个项目哪些地方值得学，哪些地方不要照搬

值得学习

- Lexer → Parser → 运行时数据的分层。
- Resource 与表现节点分离。
- 通过 ID 查询角色和背景。
- 用场景组合文本框、人物、背景和音乐。
- 用信号和 `await` 编写顺序剧情。
- 把选项 UI 做成单独组件。
- 让 `Main` 管理剧情段，`ScenePlayer` 管理单段播放。

需要批判性阅读

1. `ScenePlayer` 仍然偏大

它同时执行：

- 剧情流程

- 背景
- 音乐
- 角色
- 转场
- 变量
- 条件
- 选择

所以它是一个很好的原型，但不是你未来所有玩法都应该继续堆进去的地方。

2. 访问了其他对象的私有成员

例如：

```
_character_displayer._left_sprite
_character_displayer._right_sprite
```

这破坏了 `CharacterDisplayer` 的封装。更合理的是提供公开接口：

```
capture_state()
restore_state()
```

3. 存变量时动态编译 GDScript

[Variables.gd:53 (</D:/projects/godot-projects/Resources/Code/godot-4-2d-visual-novel/godot/Autoloads/Variables.gd:53>) 使用动态生成 GDScript 来计算变量值。

这种方式：

- 不利于类型检查
- 错误难定位
- 对不可信剧情内容存在安全风险
- 很难控制表达式支持范围

你的 DrameX 应实现受限制的表达式求值器，不应执行任意 GDScript。

4. 用三个 `process_frame` 解决 UI 时序

`ScenePlayer` 中连续等待三帧是临时修补：

```
await get_tree().process_frame
await get_tree().process_frame
await get_tree().process_frame
```

学习时应该追问真正原因，而不是把这种技巧复制到自己的代码里。

5. Transpiler 使用超大数字区分分子树

它使用十亿和二十一亿作为选择、条件节点偏移量。这可以工作，但不是很稳健的长期节点 ID 方案。你的 DrameX 可以使用：

- 单调递增的节点 ID
- 每个编译单元独立分配
- 显式边关系

- 稳定的源码位置与调试信息

6. 存在明显的小错误

[Character.gd:31 (</D:/projects/godot-projects/Resources/Code/godot-4-2d-visual-novel/godot/Characters/Character.gd:31>) 的年龄 setter 使用：

```
age = int(min(value, 0))
```

这会把正数年龄压成 `0`，通常应当是 `max(value, 0)`。

这说明该项目适合学习设计思路，但不能把所有实现都视为标准答案。

十、最适合你的学习计划

第 1 周：运行时和场景组织

目标：

- 能解释 `Main` → `ScenePlayer` → `TextBox` 调用链。
- 理解信号、`await` 和子场景职责。
- 能添加角色、背景和简单剧情。
- 不阅读 `Lexer/Parser` 内部细节。

练习成果：

一段包含两个角色、两个背景、一次选择和一次跳转的小剧情

第 2 周：数据驱动

目标：

- 理解 `Character`、`Background`、`Song` Resource。
- 理解 `ResourceDB` 的 ID 映射。
- 对比你项目里的 `Character.CHARACTER_DETAILS`。
- 设计你自己的 `CharacterDefinition`，但暂时不用实际重构。

练习成果：

画出“剧情角色 ID → ResourceDB → Character Resource → CharacterDisplayer”的流程

第 3 周：Lexer 与 Parser

目标：

- 能区分 `Token`、`SyntaxTree` 和运行时节点。
- 能手工分析一条对白和一个 choice。
- 能为一个简单新命令设计 `Token` 和语法节点。

建议练习命令：

```
wait 1.0
```

先只写设计：

```
Lexer 识别什么？
Parser 产生什么 Expression？
Transpiler 产生什么 Node？
Player 如何执行？
```

暂时不要求实际实现。

第 4 周：实现一个最小命令

选择 `wait` 最合适，因为它不需要新美术或复杂 UI：

```
wait 1.0
```

需要贯穿修改四层：

```
SceneLexer
→ SceneParser
→ SceneTranspiler.WaitCommandNode
→ ScenePlayer await timer
```

完成这个练习后，你就真正理解了整套架构。

第 5 周：映射到 DrameX

把项目的概念映射到你的设计：

示例项目	你的 DrameX
SceneLexer	DrameXLexer
SceneParser	DrameXParser
SyntaxTree	DrameX AST
SceneTranspiler	DrameXCompiler
DialogueTree	StoryProgram
BaseNode	StoryAction
ScenePlayer	StoryRunner
ResourceDB	ContentRegistry
Variables	GameState
TextBox 等	PresentationDirector 管理的视图

你不应该直接基于该项目继续扩大 `ScenePlayer`，而应吸收它的编译流水线，再进一步把“运行逻辑”和“表现协调”分离。

十一、阅读时始终问这五个问题

针对每个类都写下：

1. 它接收什么输入？
2. 它产生什么输出或信号？
3. 它保存什么状态？
4. 它依赖谁？
5. 修改它时，哪些其他模块会被迫修改？

例如 `TextBox`：

```
输入：文本、角色名、选择列表
输出：next_requested、choice_made
状态：当前文字 Tween、箭头可见状态
依赖：自身子节点、ResourceDB
职责：显示对白并接收阅读操作
```

再继续追问：

`TextBox` 是否真的应该依赖 `ResourceDB`？

答案很可能是否定的。上层已经可以直接告诉它“是否为旁白”。这类问题正是学习低耦合设计最有效的方法。

最终，你应该能独立画出下面这条链，并解释每一层为什么存在：

```
test-scene.txt
  ↓ tokenize
Token[]
  ↓ parse
SyntaxTree
  ↓ transpile
DialogueTree
  ↓ run_scene
ScenePlayer
  ↓ method + signal + await
TextBox / CharacterDisplayer / Background / BGM
```

当你能在不查看代码的情况下讲清这条链，并能独立增加一个 `wait` 命令时，就已经学到了这个项目最有价值的部分。